



INTAKE 46

Infrastructure as Code

3-Tier Web Application on AWS

Terraform & Ansible

Track: System Administration



Contents

1	Introduction	3
1.1	Tools Used	3
1.2	Application Architecture Overview	3
2	Infrastructure Overview	3
2.1	Architecture Diagram	4
3	HashiCorp Vault Integration	4
3.1	Phase 1: Environment Preparation & Vault Reset	5
3.1.1	Clear Existing Variables	5
3.1.2	Define Vault Server Address	5
3.1.3	Reset Vault Data (Clean Slate)	5
3.2	Phase 2: Initialization and Unsealing	5
3.2.1	Initialize Vault	5
3.2.2	Inject Root Token into Terraform Variables	5
3.2.3	Unseal the Vault	6
3.3	Phase 3: Authentication and Secret Storage	6
3.3.1	Log In to Vault	6
3.3.2	Enable the KV Secrets Engine	6
3.3.3	Store AWS Credentials in Vault	6
3.4	Phase 4: Verification	6
3.4.1	CLI Verification	6
3.4.2	Web UI Verification	7
4	Network Infrastructure	7
4.1	Terraform Configuration	7
4.1.1	main.tf	7
4.1.2	variables.tf	8
4.1.3	outputs.tf	9
4.2	Deployed Infrastructure	9
4.3	Verification Screenshots	9
4.3.1	VPC	10
4.3.2	Internet Gateway	10
4.3.3	Subnets	11
4.3.4	NAT Gateway	11
5	Security Groups	11
5.1	Terraform Configuration	12
5.1.1	main.tf	12
5.1.2	variables.tf	14
5.1.3	outputs.tf	15
5.2	Deployed Security Groups	15
5.3	Verification Screenshot	15

6	Load Balancers	16
6.1	Terraform Configuration	16
6.1.1	main.tf	16
6.1.2	variables.tf	18
6.1.3	outputs.tf	18
6.2	Deployed Load Balancers	18
6.3	Verification Screenshots	19
6.3.1	Load Balancers Overview	19
6.3.2	Target Groups	19
7	Compute Infrastructure	20
7.1	Terraform Configuration	20
7.1.1	main.tf	20
7.1.2	variables.tf	22
7.1.3	outputs.tf	23
7.2	Deployed Compute Infrastructure	23
7.3	Verification Screenshots	24
7.3.1	EC2 Instances	24
7.3.2	Auto Scaling Groups	25
8	Ansible Configuration	25
8.1	Dynamic Inventory	25
8.1.1	aws_ec2.yml	25
8.1.2	ansible.cfg	26
8.1.3	group_vars/role_frontend.yml	26
8.2	Main Playbook	26
8.3	Database Role	27
8.3.1	templates/mongod.conf.j2	27
8.3.2	tasks/main.yml	27
8.3.3	handlers/main.yml	28
8.4	Backend Role	29
8.4.1	tasks/main.yml	29
8.5	Frontend Role	30
8.5.1	templates/nginx_conf.j2	30
8.5.2	tasks/main.yml	30
8.5.3	handlers/main.yml	31
8.6	Verification Screenshots	31
8.6.1	Playbook Execution Summary	32
9	End-to-End Testing	32
9.1	Test 1: Application Loads Successfully	32
9.2	Test 2: Waitlist Signup Works End-to-End	33
9.3	Test 3: Data Replicated Across All MongoDB Nodes	34
9.4	Summary	35

1 Introduction

Modern software applications are increasingly built using a **three-tier architecture**, which separates concerns into three distinct layers: a **Presentation Layer** (Frontend), an **Application Layer** (Backend), and a **Data Layer** (Database). This separation improves scalability, maintainability, and security by ensuring that each layer can be managed, scaled, and secured independently.

Deploying and managing such an architecture manually is error-prone and time-consuming, especially at scale. This is where **Infrastructure as Code (IaC)** becomes essential. IaC is the practice of defining and provisioning infrastructure through machine-readable configuration files rather than through manual processes or interactive tools. It brings the same discipline of versioning, peer review, and automation that developers apply to application code – to infrastructure itself.

1.1 Tools Used

This project leverages two industry-standard IaC tools:

- **Terraform** – An open-source tool by HashiCorp used to *provision* cloud infrastructure declaratively. It communicates with the AWS API to create and manage resources such as VPCs, subnets, EC2 instances, security groups, load balancers, and RDS databases. Terraform maintains a *state file* that tracks the real-world status of every managed resource.
- **Ansible** – An open-source automation tool by Red Hat used to *configure* the servers that Terraform provisions. Once the machines exist, Ansible connects over SSH and runs *playbooks* – sets of ordered tasks – to install software, deploy application code, and manage services on the Frontend, Backend, and Database tiers.

1.2 Application Architecture Overview

The three-tier web application deployed in this project is structured as follows:

- **Frontend (Presentation Layer)** – A web server (e.g. Nginx) that serves the user-facing interface. It sits in a *public subnet* and is accessible from the internet through a Load Balancer. It forwards API requests to the Backend layer.
- **Backend (Application Layer)** – An application server that processes business logic and handles communication between the Frontend and the Database. It resides in a *private subnet*, shielded from direct internet access, and is only reachable from the Frontend tier.
- **Database (Data Layer)** – A managed database deployed in a *private subnet*. It stores all persistent application data and is only accessible from the Backend tier, ensuring maximum security.

By combining **Terraform** for provisioning and **Ansible** for configuration, this project demonstrates a complete, automated, and repeatable pipeline for deploying a production-grade three-tier web application on **Amazon Web Services (AWS)**.

2 Infrastructure Overview

The infrastructure for this project is hosted entirely on **Amazon Web Services (AWS)** and is organized inside a custom **Virtual Private Cloud (VPC)** to provide full network isolation and control.

The architecture follows AWS best practices for high availability and security: all public-facing resources are placed in **public subnets** behind an **Application Load Balancer (ALB)**, while the Backend and Database tiers are placed in **private subnets** with no direct internet exposure. An **Internet Gateway** provides outbound internet access for the public subnet, and a **NAT Gateway** allows private instances to reach the internet for package installation without being directly reachable from outside.

Security Groups act as virtual firewalls at the instance level, enforcing strict inbound and outbound traffic rules between each tier. Only the minimum required ports are open between layers, following the principle of least privilege.

The full infrastructure – including the VPC, subnets, routing tables, security groups, EC2 instances, load balancer, and RDS database – is defined entirely in **Terraform** configuration files and can be provisioned or destroyed with a single command.

2.1 Architecture Diagram

The diagram below illustrates the complete AWS infrastructure layout, showing how the three tiers interact within the VPC and how traffic flows from the internet through to the Database.

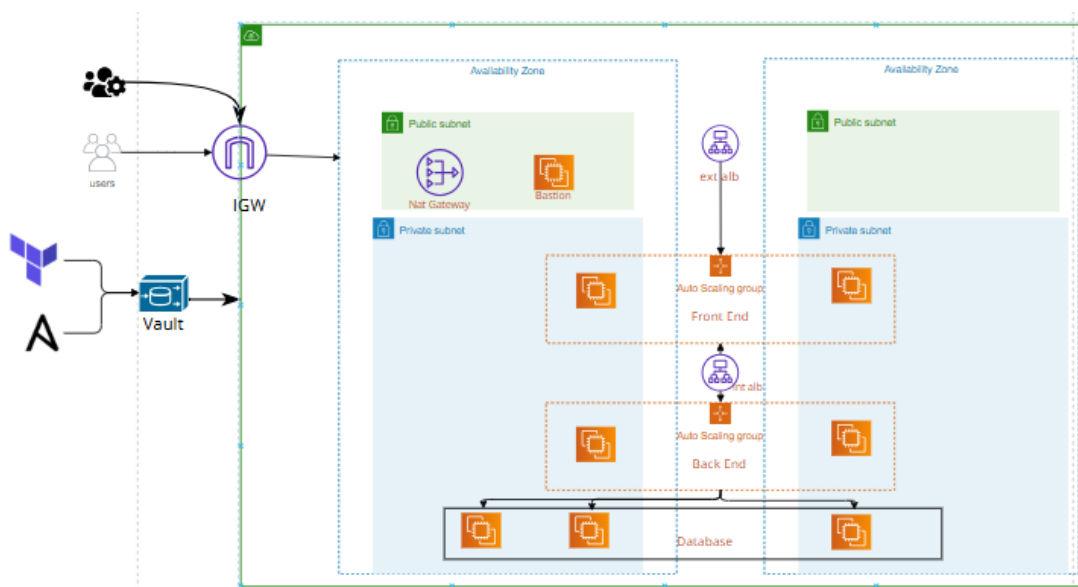


Figure 1: AWS 3-Tier Web Application – Infrastructure Architecture Diagram

3 HashiCorp Vault Integration

HashiCorp Vault is an open-source secrets management tool designed to securely store, access, and manage sensitive data such as API keys, passwords, certificates, and cloud credentials. Rather than hard-coding secrets into configuration files or environment variables – a practice that creates serious security risks – Vault acts as a centralized, encrypted secrets store with fine-grained access control policies and a full audit log.

In this project, Vault serves as the *single source of truth* for AWS credentials. Both **Terraform** and **Ansible** are configured to retrieve secrets dynamically from Vault at runtime, ensuring that no sensitive credentials are ever written to disk or committed to version control.

- **Terraform** reads the AWS credentials (Access Key, Secret Key, and Session Token) directly

from Vault using the `vault` provider, allowing infrastructure to be provisioned without exposing secrets in `terraform.tfvars` in plain text.

- **Ansible** retrieves secrets from Vault at playbook execution time using the `hashi_vault` lookup plugin, injecting credentials into tasks dynamically without storing them in inventory files or group variables.

3.1 Phase 1: Environment Preparation & Vault Reset

Before starting, all environment variables tied to existing Vault tokens and AWS credentials were cleared, and the Vault server was explicitly reset to provide a clean slate.

3.1.1 Clear Existing Variables

```
1 unset VAULT_TOKEN
2 unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN
```

3.1.2 Define Vault Server Address

```
1 export VAULT_ADDR="https://127.0.0.1:8200"
2 export VAULT_SKIP_VERIFY=true
```

Note: VAULT_SKIP_VERIFY

`VAULT_SKIP_VERIFY=true` is set because the local Vault server is using a self-signed TLS certificate. This should **not** be used in production environments.

3.1.3 Reset Vault Data (Clean Slate)

```
1 systemctl stop vault
2 rm -rf /opt/vault/data/*
3 systemctl start vault
```

3.2 Phase 2: Initialization and Unsealing

With a fresh Vault instance running, the server must be initialized to generate the Master Keys and the Initial Root Token, then unsealed before it can serve secrets.

3.2.1 Initialize Vault

```
1 vault operator init
```

This command produces **5 Unseal Keys** and **1 Initial Root Token**. The key threshold is set to **3**, meaning at least 3 of the 5 unseal keys must be provided to bring the Vault out of its sealed state.

3.2.2 Inject Root Token into Terraform Variables

The newly generated Root Token was added to `terraform.tfvars` so that Terraform can authenticate with Vault during infrastructure provisioning:

```
1 vim terraform.tfvars
2 # Updated value:
3 vault_token = "hvs.Zxdbgn2wdiyt9QirWVt3I9W1"
```

3.2.3 Unseal the Vault

The following command was executed three times, each time supplying a different unseal key, until the Vault status transitioned from **Sealed: true** to **Sealed: false**:

```
1 vault operator unseal eEOB/0VjbB43EvIs0n0WGbf02bM0/SGMQT1wl+0AJsmJ
```

3.3 Phase 3: Authentication and Secret Storage

Once unsealed, the root user authenticated to Vault, enabled the secrets engine, and stored the AWS sandbox credentials as a single secret payload.

3.3.1 Log In to Vault

```
1 vault login hvs.Zxdbgn2wdiyt9QirWVt3I9W1
```

3.3.2 Enable the KV Secrets Engine

```
1 vault secrets enable -path=secret kv-v2
```

This enables the **Key/Value version 2** secrets engine at the **secret/** path, which supports secret versioning and metadata tracking.

3.3.3 Store AWS Credentials in Vault

The AWS temporary sandbox credentials were injected into Vault as a single structured secret payload under the path **secret/aws/creds**:

```
1 vault kv put secret/aws/creds \  
2   access_key="ASIAXR4VKVAXHLRHWE2" \  
3   secret_key="kCOQ0gp/hymUN5MfyFbDRvaSqCSNMNWTkadyM5hg" \  
4   session_token="IQoJb3JpZ2... [truncated for readability] ...  
   ZlQY31hEJOAQkE"
```

3.4 Phase 4: Verification

The stored credentials were verified through both the CLI and the Vault Web UI to confirm successful secret creation.

3.4.1 CLI Verification

```
1 vault kv get secret/aws/creds
```

This command returns both the metadata and the actual key-value pairs stored in the previous phase, confirming that **access_key**, **secret_key**, and **session_token** are correctly saved.

3.4.2 Web UI Verification

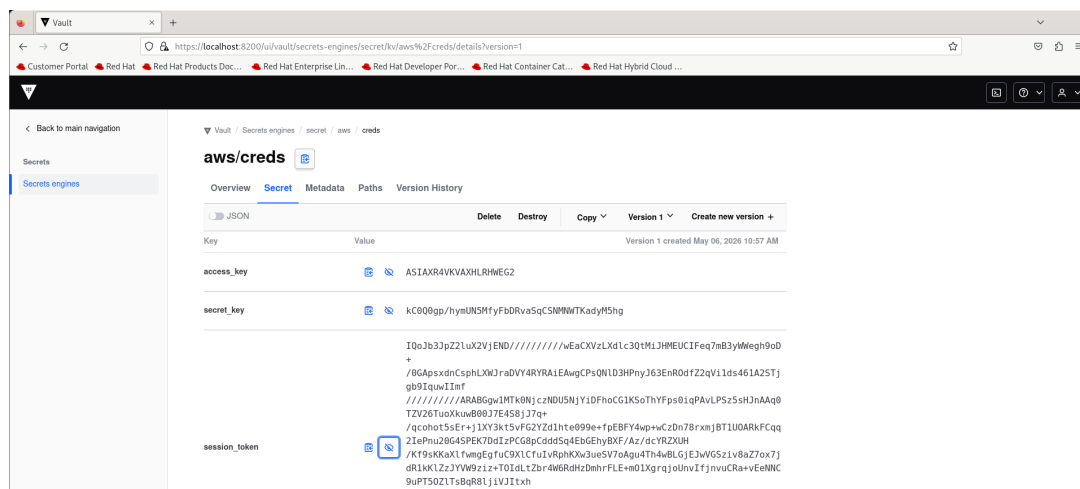


Figure 2: HashiCorp Vault Web UI verification showing successful initialization.

4 Network Infrastructure

The network layer forms the foundation of the entire three-tier architecture. All compute, load balancing, and database resources are deployed inside a single **Virtual Private Cloud (VPC)**, which provides full logical isolation from other AWS accounts and gives the team complete control over IP addressing, routing, and traffic flow. The network is defined entirely in Terraform and can be provisioned or torn down with a single command.

4.1 Terraform Configuration

The network module is split across three standard Terraform files: `main.tf`, `variables.tf`, and `outputs.tf`. Together they declare every network resource – from the VPC itself down to individual route table associations – in a fully parameterized, reusable form.

4.1.1 main.tf

The main configuration file provisions all network primitives in the following order: VPC, Internet Gateway, public and private subnets, Elastic IP, NAT Gateway, and the route tables with their subnet associations.

```

1 resource "aws_vpc" "main" {
2   cidr_block           = var.vpc_cidr
3   enable_dns_hostnames = true
4   enable_dns_support   = true
5   tags = { Name = "${var.project_name}-vpc" }
6 }
7
8 resource "aws_internet_gateway" "igw" {
9   vpc_id = aws_vpc.main.id
10  tags   = { Name = "${var.project_name}-igw" }
11 }
12
13 resource "aws_subnet" "public" {
14   count
    
```



```

15     vpc_id                = aws_vpc.main.id
16     cidr_block             = var.public_cidrs[count.index]
17     availability_zone      = var.azs[count.index]
18     map_public_ip_on_launch = true
19     tags = { Name = "${var.project_name}-public-${count.index + 1}" }
20 }
21
22 resource "aws_subnet" "private" {
23     count                = 2
24     vpc_id              = aws_vpc.main.id
25     cidr_block          = var.private_cidrs[count.index]
26     availability_zone    = var.azs[count.index]
27     tags = { Name = "${var.project_name}-private-${count.index + 1}" }
28 }
29
30 resource "aws_eip" "nat" {
31     domain = "vpc"
32 }
33
34 resource "aws_nat_gateway" "main" {
35     allocation_id = aws_eip.nat.id
36     subnet_id     = aws_subnet.public[0].id
37     tags = { Name = "${var.project_name}-nat" }
38 }
39
40 resource "aws_route_table" "public_rt" {
41     vpc_id = aws_vpc.main.id
42     route {
43         cidr_block = "0.0.0.0/0"
44         gateway_id = aws_internet_gateway.igw.id
45     }
46 }
47
48 resource "aws_route_table_association" "public_assoc" {
49     count                = 2
50     subnet_id           = aws_subnet.public[count.index].id
51     route_table_id      = aws_route_table.public_rt.id
52 }
53
54 resource "aws_route_table" "private_rt" {
55     vpc_id = aws_vpc.main.id
56     route {
57         cidr_block      = "0.0.0.0/0"
58         nat_gateway_id = aws_nat_gateway.main.id
59     }
60 }
61
62 resource "aws_route_table_association" "private_assoc" {
63     count                = 2
64     subnet_id           = aws_subnet.private[count.index].id
65     route_table_id      = aws_route_table.private_rt.id
66 }

```

4.1.2 variables.tf

All configurable values are extracted into input variables so that the same module can be reused across different environments simply by supplying a different `terraform.tfvars` file.

```

1 variable "project_name" {}
2 variable "vpc_cidr"      { default = "10.0.0.0/16" }
3 variable "public_cidrs"  { type = list(string) }
4 variable "private_cidrs" { type = list(string) }
5 variable "azs"           { type = list(string) }

```

4.1.3 outputs.tf

The VPC ID and subnet IDs are exported as Terraform outputs so that downstream modules – such as the Security Groups and EC2 modules – can reference them without duplicating resource lookups.

```

1 output "vpc_id" {
2   value = aws_vpc.main.id
3 }
4
5 output "public_subnet_ids" {
6   value = aws_subnet.public[*].id
7 }
8
9 output "private_subnet_ids" {
10  value = aws_subnet.private[*].id
11 }

```

4.2 Deployed Infrastructure

After running `terraform apply`, the following resources were successfully created and verified in the AWS Console. The table below summarizes the complete deployed network configuration for the `novolt` project.

Resource	Name	ID / Value	Detail
VPC	novolt-vpc	vpc-06c3ce08596c4024a	CIDR: 10.0.0.0/16
Internet Gateway	novolt-igw	igw-0d9bac87cda1ba29b	Attached to VPC
NAT Gateway	novolt-nat	nat-03696ee806ad947d2	Public IP: 98.91.63.242

Subnet Name	Type	CIDR	Availability Zone	Subnet ID
novolt-public-1	Public	10.0.1.0/24	us-east-1a	subnet-081fe3e0f775b8768
novolt-public-2	Public	10.0.2.0/24	us-east-1b	subnet-0447d65e264823013
novolt-private-1	Private	10.0.3.0/24	us-east-1a	subnet-003eb95cd36873698
novolt-private-2	Private	10.0.4.0/24	us-east-1b	subnet-06cc9375230352635

The two public subnets span `us-east-1a` and `us-east-1b` to ensure high availability for the Frontend tier and the Application Load Balancer. The two private subnets mirror the same Availability Zones, hosting the Backend and Database tiers with no direct internet exposure. The NAT Gateway is placed in `novolt-public-1` and holds the Elastic IP `98.91.63.242`, allowing private instances to initiate outbound connections for package installation without being reachable from the internet.

4.3 Verification Screenshots

The following screenshots were captured from the AWS Management Console to confirm that all network resources were provisioned correctly by Terraform.

4.3.1 VPC

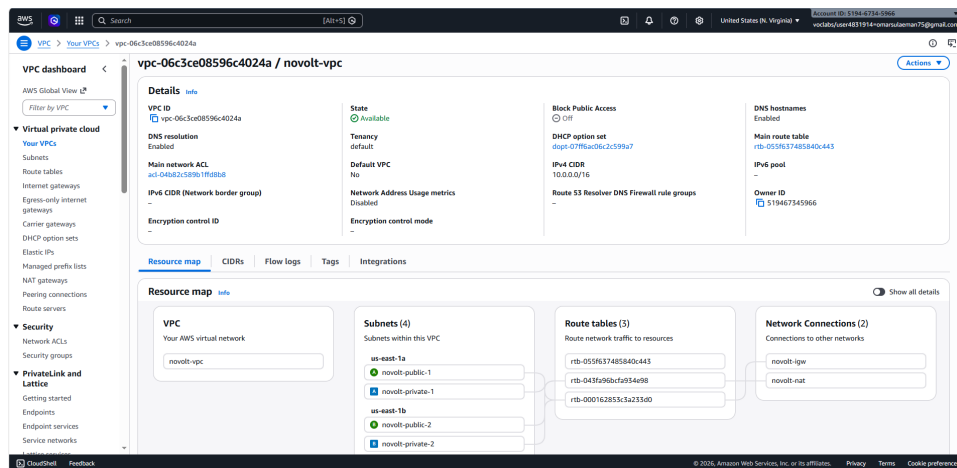


Figure 3: AWS Console – VPC Dashboard: novolt-vpc (vpc-06c3ce08596c4024a) with CIDR 10.0.0.0/16, DNS hostnames and DNS resolution Enabled

4.3.2 Internet Gateway

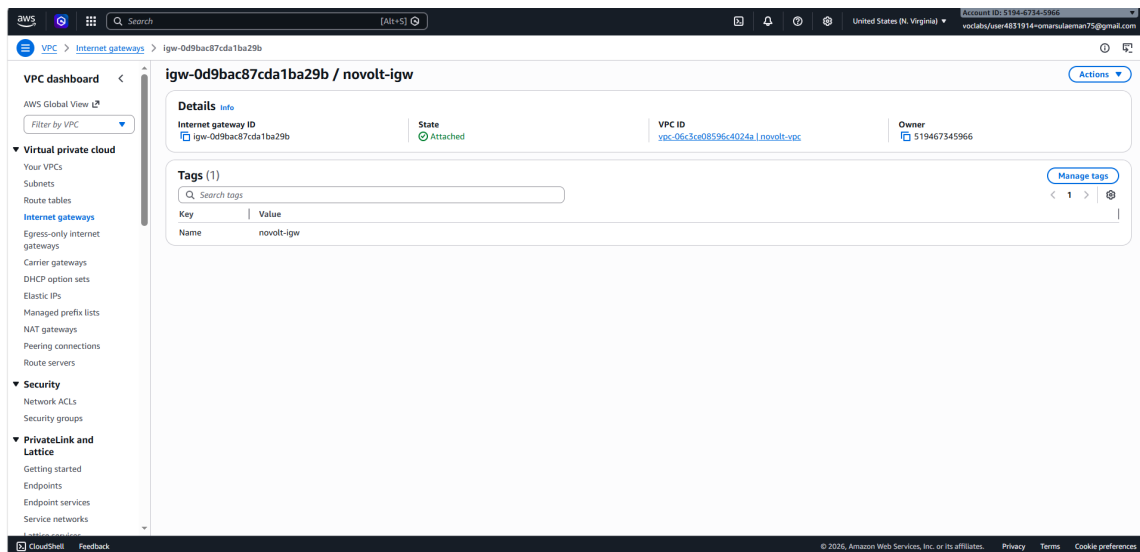


Figure 4: AWS Console – Internet Gateway: novolt-igw (igw-0d9bac87cda1ba29b) attached to vpc-06c3ce08596c4024a

4.3.3 Subnets

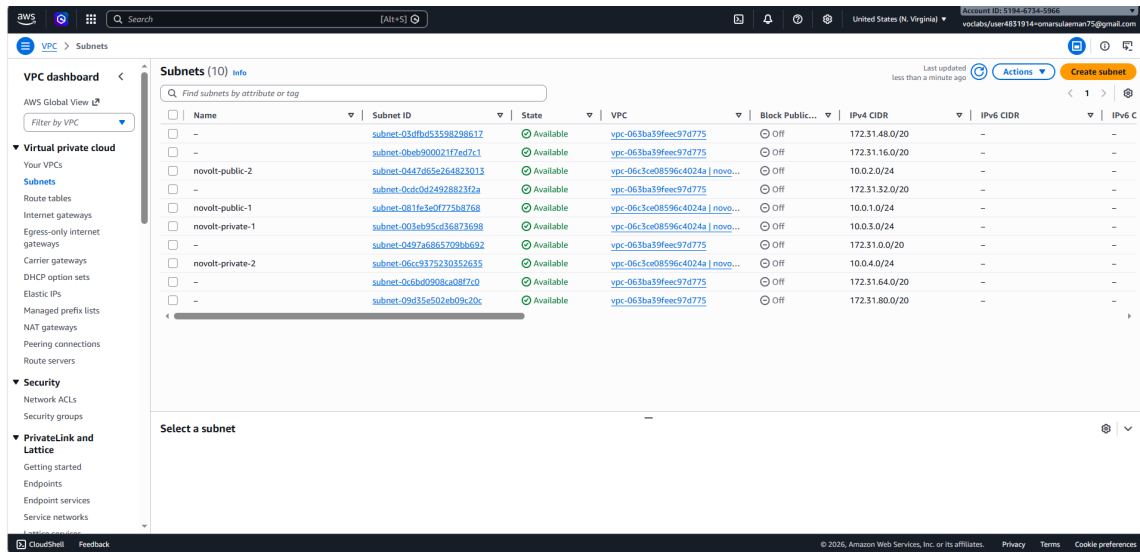


Figure 5: AWS Console – Subnets: all four subnets (`novolt-public-1`, `novolt-public-2`, `novolt-private-1`, `novolt-private-2`) with their CIDRs and Availability Zones

4.3.4 NAT Gateway

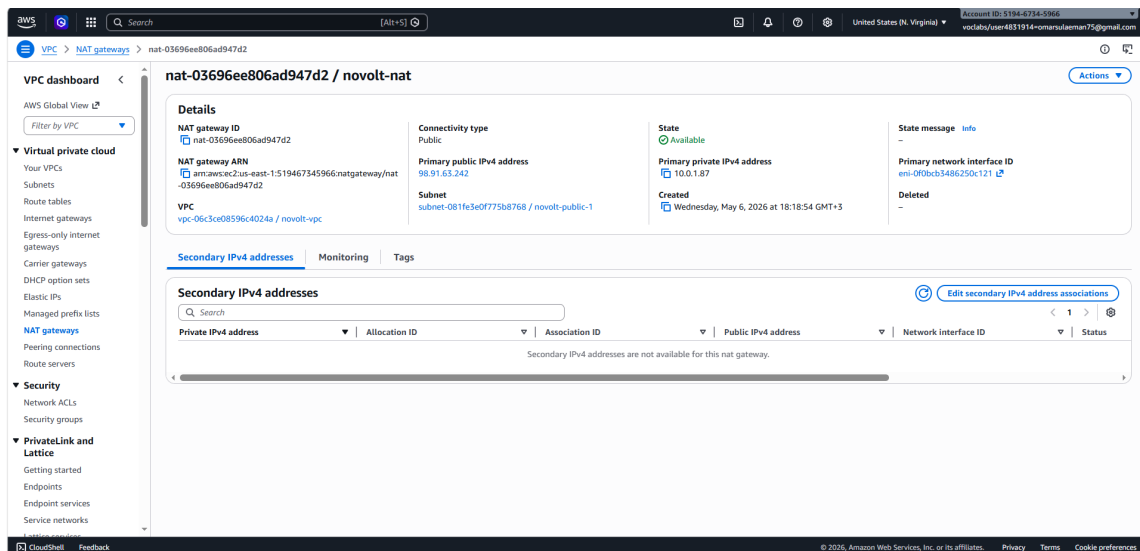


Figure 6: AWS Console – NAT Gateway: `novolt-nat` (`nat-03696ee806ad947d2`), state Available, Public IP 98.91.63.242, Private IP 10.0.1.87, located in subnet-081fe3e0f775b8768

5 Security Groups

Security Groups act as stateful virtual firewalls at the instance level, controlling all inbound and outbound traffic for every resource in the VPC. This project follows the **principle of least privilege** – each tier only accepts traffic from the specific resource directly upstream of it, and all inter-tier rules reference Security Group IDs rather than CIDR blocks, making the rules dynamic and tamper-resistant.

Six Security Groups are defined in total, forming a strict traffic chain: Internet → External ALB → Frontend → Internal ALB → Backend → Database. A dedicated Bastion Security Group is also created to allow controlled SSH access to all private instances from a single hardened entry point.

5.1 Terraform Configuration

The Security Groups module follows the same three-file structure used in the Network module: `main.tf` declares all six groups and their rules, `variables.tf` accepts the `project_name` and `vpc_id` as inputs, and `outputs.tf` exports every Security Group ID so that downstream modules can reference them without duplication.

5.1.1 main.tf

Each Security Group is declared with explicit ingress and egress rules. All egress blocks allow full outbound traffic (0.0.0.0/0) so that instances can reach the internet for package updates via the NAT Gateway. Ingress rules are locked to the minimum required ports and sources.

```

1 resource "aws_security_group" "bastion_sg" {
2   name           = "${var.project_name}-bastion-sg"
3   description    = "Allow SSH from anywhere to Bastion"
4   vpc_id         = var.vpc_id
5
6   ingress {
7     from_port     = 22
8     to_port       = 22
9     protocol      = "tcp"
10    cidr_blocks    = ["0.0.0.0/0"]
11  }
12  egress {
13    from_port      = 0
14    to_port        = 0
15    protocol       = "-1"
16    cidr_blocks    = ["0.0.0.0/0"]
17  }
18 }
19
20 resource "aws_security_group" "ext_alb_sg" {
21   name           = "${var.project_name}-ext-alb-sg"
22   description    = "Allow HTTP/HTTPS from Internet"
23   vpc_id         = var.vpc_id
24
25   ingress {
26     from_port     = 80
27     to_port       = 80
28     protocol      = "tcp"
29     cidr_blocks    = ["0.0.0.0/0"]
30   }
31   egress {
32     from_port      = 0
33     to_port        = 0
34     protocol       = "-1"
35     cidr_blocks    = ["0.0.0.0/0"]
36   }
37 }
38
39 resource "aws_security_group" "frontend_sg" {

```

```

40     name          = "${var.project_name}-frontend-sg"
41     description   = "Allow traffic ONLY from External ALB"
42     vpc_id        = var.vpc_id
43
44     ingress {
45         from_port    = 80
46         to_port      = 80
47         protocol     = "tcp"
48         security_groups = [aws_security_group.ext_alb_sg.id]
49     }
50     ingress {
51         from_port    = 22
52         to_port      = 22
53         protocol     = "tcp"
54         security_groups = [aws_security_group.bastion_sg.id]
55     }
56     egress {
57         from_port    = 0
58         to_port      = 0
59         protocol     = "-1"
60         cidr_blocks  = ["0.0.0.0/0"]
61     }
62 }
63
64 resource "aws_security_group" "int_alb_sg" {
65     name          = "${var.project_name}-int-alb-sg"
66     description   = "Allow traffic ONLY from Frontend servers"
67     vpc_id        = var.vpc_id
68
69     ingress {
70         from_port    = 80
71         to_port      = 80
72         protocol     = "tcp"
73         security_groups = [aws_security_group.frontend_sg.id]
74     }
75     ingress {
76         from_port    = 3000
77         to_port      = 3000
78         protocol     = "tcp"
79         security_groups = [aws_security_group.frontend_sg.id]
80     }
81     egress {
82         from_port    = 0
83         to_port      = 0
84         protocol     = "-1"
85         cidr_blocks  = ["0.0.0.0/0"]
86     }
87 }
88
89 resource "aws_security_group" "backend_sg" {
90     name          = "${var.project_name}-backend-sg"
91     description   = "Allow traffic ONLY from Internal ALB"
92     vpc_id        = var.vpc_id
93
94     ingress {
95         from_port    = 3000
96         to_port      = 3000

```

```

97     protocol      = "tcp"
98     security_groups = [aws_security_group.int_alb_sg.id]
99   }
100   ingress {
101     from_port      = 22
102     to_port        = 22
103     protocol       = "tcp"
104     security_groups = [aws_security_group.bastion_sg.id]
105   }
106   egress {
107     from_port      = 0
108     to_port        = 0
109     protocol       = "-1"
110     cidr_blocks    = ["0.0.0.0/0"]
111   }
112 }
113
114 resource "aws_security_group" "db_sg" {
115   name          = "${var.project_name}-db-sg"
116   description   = "Allow traffic ONLY from Backend and DB replication"
117   vpc_id        = var.vpc_id
118
119   ingress {
120     from_port      = 27017
121     to_port        = 27017
122     protocol       = "tcp"
123     security_groups = [aws_security_group.backend_sg.id]
124   }
125   ingress {
126     from_port      = 27017
127     to_port        = 27017
128     protocol       = "tcp"
129     self           = true
130   }
131   ingress {
132     from_port      = 22
133     to_port        = 22
134     protocol       = "tcp"
135     security_groups = [aws_security_group.bastion_sg.id]
136   }
137   egress {
138     from_port      = 0
139     to_port        = 0
140     protocol       = "-1"
141     cidr_blocks    = ["0.0.0.0/0"]
142   }
143 }

```

5.1.2 variables.tf

The module accepts two inputs: the project name used for consistent resource tagging, and the VPC ID into which all Security Groups are created.

```

1 variable "project_name" {}
2 variable "vpc_id"       {}

```

5.1.3 outputs.tf

All six Security Group IDs are exported so that the EC2, ALB, and RDS modules can reference them directly without hard-coding any resource identifiers.

```

1 output "bastion_sg_id" {
2   value      = aws_security_group.bastion_sg.id
3   description = "Security Group ID for the Bastion Host"
4 }
5 output "ext_alb_sg_id" {
6   value      = aws_security_group.ext_alb_sg.id
7   description = "Security Group ID for the External ALB"
8 }
9 output "frontend_sg_id" {
10  value      = aws_security_group.frontend_sg.id
11  description = "Security Group ID for the Frontend EC2 instances"
12 }
13 output "int_alb_sg_id" {
14  value      = aws_security_group.int_alb_sg.id
15  description = "Security Group ID for the Internal ALB"
16 }
17 output "backend_sg_id" {
18  value      = aws_security_group.backend_sg.id
19  description = "Security Group ID for the Backend EC2 instances"
20 }
21 output "db_sg_id" {
22  value      = aws_security_group.db_sg.id
23  description = "Security Group ID for the Database EC2 instances"
24 }

```

5.2 Deployed Security Groups

After running `terraform apply`, all six Security Groups were created inside VPC `vpc-06c3ce08596c4024a`. The table below summarises each group, its ID, and the inbound rules enforced.

Name	Security Group ID	Port(s)	Source
novolt-bastion-sg	sg-00a2d1c5ced7dd2c1	22	0.0.0.0/0 (Internet)
novolt-ext-alb-sg	sg-0b3459561c6c0b41d	80	0.0.0.0/0 (Internet)
novolt-frontend-sg	sg-0744f55ff537dbf56	80, 22	ext-alb-sg, bastion-sg
novolt-int-alb-sg	sg-03394fb6008eefafe	80, 3000	frontend-sg
novolt-backend-sg	sg-04c4ad256bb98e276	3000, 22	int-alb-sg, bastion-sg
novolt-db-sg	sg-056a0c1f0eab30c62	27017, 22	backend-sg, self, bastion-sg

The `self = true` rule on `novolt-db-sg` port 27017 allows MongoDB replica set members within the same Security Group to communicate with each other for replication without opening that port to any other resource.

5.3 Verification Screenshot

The following screenshot was captured from the AWS Management Console to confirm that all six Security Groups were provisioned correctly by Terraform.

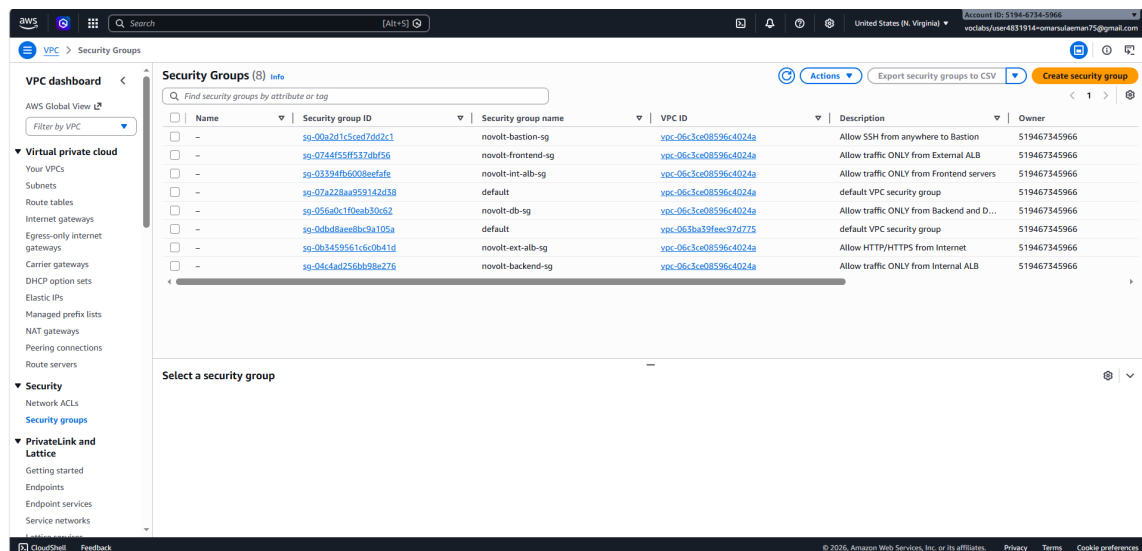


Figure 7: AWS Console – Security Groups: all six novolt Security Groups attached to vpc-06c3ce08596c4024a with their respective IDs and descriptions

6 Load Balancers

Two Application Load Balancers (ALBs) are deployed to manage and distribute traffic between the tiers of the architecture. The **External ALB** sits in the public subnets and serves as the single entry point for all internet traffic, forwarding requests to the Frontend EC2 instances. The **Internal ALB** sits in the private subnets and acts as the gateway between the Frontend and Backend tiers, ensuring that the Backend is never directly reachable from the internet. Both ALBs perform continuous health checks on their registered targets and only route traffic to healthy instances.

6.1 Terraform Configuration

The Load Balancer module follows the same three-file structure used across all other modules. `main.tf` declares both ALBs, their target groups, and their listeners. `variables.tf` accepts the VPC ID, subnet IDs, and Security Group IDs as inputs from the Network and Security Group modules. `outputs.tf` exports the ALB DNS names and target group ARNs so that the EC2 module can register instances automatically.

6.1.1 main.tf

The External ALB is created with `internal = false` and attached to the public subnets, while the Internal ALB uses `internal = true` and is placed in the private subnets. Each ALB has a dedicated target group and an HTTP listener that forwards all traffic to that group.

```
1 resource "aws_lb" "external_alb" {
2   name           = "${var.project_name}-ext-alb"
3   internal       = false
4   load_balancer_type = "application"
5   security_groups = [var.ext_alb_sg_id]
6   subnets       = var.public_subnet_ids
7   tags = { Name = "${var.project_name}-ext-alb" }
8 }
9
```

```
10 resource "aws_lb_target_group" "frontend_tg" {
11     name      = "${var.project_name}-fe-tg"
12     port      = 80
13     protocol  = "HTTP"
14     vpc_id    = var.vpc_id
15
16     health_check {
17         path      = "/"
18         interval  = 30
19         timeout   = 5
20         healthy_threshold = 2
21         unhealthy_threshold = 2
22     }
23 }
24
25 resource "aws_lb_listener" "frontend_listener" {
26     load_balancer_arn = aws_lb.external_alb.arn
27     port              = 80
28     protocol          = "HTTP"
29
30     default_action {
31         type      = "forward"
32         target_group_arn = aws_lb_target_group.frontend_tg.arn
33     }
34 }
35
36 resource "aws_lb" "internal_alb" {
37     name            = "${var.project_name}-int-alb"
38     internal        = true
39     load_balancer_type = "application"
40     security_groups = [var.int_alb_sg_id]
41     subnets        = var.private_subnet_ids
42     tags = { Name = "${var.project_name}-int-alb" }
43 }
44
45 resource "aws_lb_target_group" "backend_tg" {
46     name      = "${var.project_name}-be-tg"
47     port      = 3000
48     protocol  = "HTTP"
49     vpc_id    = var.vpc_id
50
51     health_check {
52         path      = "/health"
53         interval  = 30
54         port      = "3000"
55     }
56 }
57
58 resource "aws_lb_listener" "backend_listener" {
59     load_balancer_arn = aws_lb.internal_alb.arn
60     port              = 80
61     protocol          = "HTTP"
62
63     default_action {
64         type      = "forward"
65         target_group_arn = aws_lb_target_group.backend_tg.arn
66     }
67 }
```

```
67 }
```

6.1.2 variables.tf

The module receives the VPC ID, both subnet ID lists, and the two Security Group IDs as inputs. These are passed in from the Network and Security Group module outputs via the root `main.tf`.

```
1 variable "project_name"      { type = string }
2 variable "vpc_id"            { type = string }
3 variable "public_subnet_ids" { type = list(string) }
4 variable "private_subnet_ids" { type = list(string) }
5 variable "ext_alb_sg_id"     { type = string }
6 variable "int_alb_sg_id"     { type = string }
```

6.1.3 outputs.tf

The DNS names of both ALBs and the ARNs of both target groups are exported. The DNS names are used to configure the Frontend application to point at the Internal ALB, and the target group ARNs are consumed by the EC2 module to register instances as targets.

```
1 output "external_alb_dns" {
2   value = aws_lb.external_alb.dns_name
3 }
4 output "internal_alb_dns" {
5   value = aws_lb.internal_alb.dns_name
6 }
7 output "frontend_tg_arn" {
8   value = aws_lb_target_group.frontend_tg.arn
9 }
10 output "backend_tg_arn" {
11   value = aws_lb_target_group.backend_tg.arn
12 }
```

6.2 Deployed Load Balancers

After running `terraform apply`, both Application Load Balancers were created and reached an **Active** state inside VPC `vpc-06c3ce08596c4024a`. The table below summarises the deployed configuration of each ALB.

Attribute	novolt-ext-alb	novolt-int-alb
State	Active	Active
Type	Application	Application
Scheme	Internet-facing	Internal
IP Address Type	IPv4	IPv4
VPC	vpc-06c3ce08596c4024a	vpc-06c3ce08596c4024a
Subnets	Public (2 AZs)	Private (2 AZs)
Security Group	sg-0b3459561c6c0b41d	sg-03394fb6008eefafe
Listener Port	80 (HTTP)	80 (HTTP)
Target Group Port	80	3000
Health Check Path	/	/health

The External ALB spans both public subnets across two Availability Zones, providing high availability for internet-facing traffic. The Internal ALB mirrors this setup across both private

subnets, ensuring that Backend traffic is equally distributed and resilient to a single Availability Zone failure. No traffic can reach the Backend directly – all requests must pass through the Frontend and then through the Internal ALB, enforcing the strict traffic chain defined by the Security Groups.

6.3 Verification Screenshots

The following screenshots were captured from the AWS Management Console to confirm that both Load Balancers and their associated target groups were provisioned correctly by Terraform.

6.3.1 Load Balancers Overview

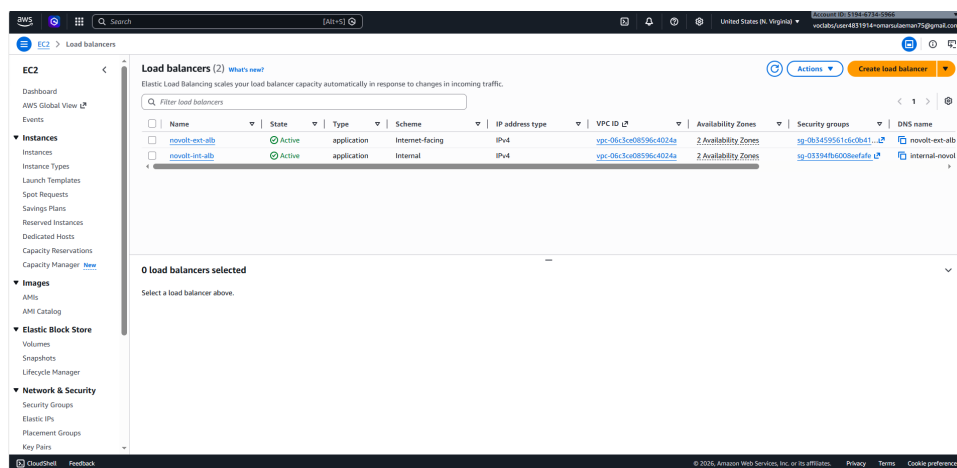


Figure 8: AWS Console – Load Balancers: `novolt-ext-alb` (Internet-facing) and `novolt-int-alb` (Internal), both in Active state inside `vpc-06c3ce08596c4024a`

6.3.2 Target Groups

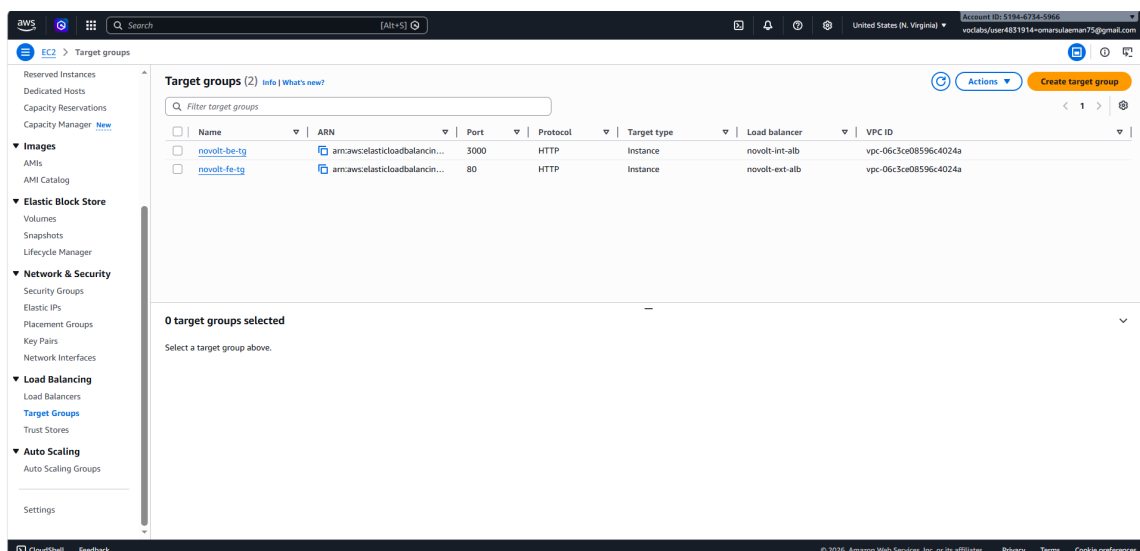


Figure 9: AWS Console – Target Groups: `novolt-fe-tg` (port 80) and `novolt-be-tg` (port 3000) with their health check configurations

7 Compute Infrastructure

The compute layer provisions all virtual machines that power the three-tier application. Every instance is deployed inside the custom VPC defined in the Network module and assigned to the appropriate Security Group defined in the Security Groups module. The compute infrastructure is split into four logical groups: a **Bastion Host** for secure SSH access, **Frontend** and **Backend** Auto Scaling Groups (ASGs) for horizontally scalable application tiers, and three dedicated **Database** instances configured as a MongoDB Replica Set for high-availability data persistence. All instances use the `t2.micro` instance type and a single parameterized AMI ID, making it straightforward to upgrade the base image across all tiers simultaneously by updating a single variable. The entire compute layer is defined in Terraform and provisioned with a single `terraform apply` command.

7.1 Terraform Configuration

The compute module follows the same three-file structure used across all other modules: `main.tf` declares all instances, launch templates, and Auto Scaling Groups; `variables.tf` accepts the subnet IDs, Security Group IDs, and key pair name as inputs from upstream modules; and `outputs.tf` exports the Bastion public IP, database private IPs, and ASG names so that downstream resources and Ansible inventory can reference them without duplication.

7.1.1 main.tf

The main configuration file provisions all compute resources in the following order: Bastion Host, Frontend Launch Template and ASG, Backend Launch Template and ASG, and the three Database EC2 instances. The ASGs are attached to their respective ALB target groups via `aws_autoscaling_attachment` resources defined in the root module.

```

1 resource "aws_instance" "bastion" {
2     ami                = var.ami_id
3     instance_type      = "t2.micro"
4     subnet_id          = var.public_subnet_ids[0]
5     vpc_security_group_ids = [var.bastion_sg_id]
6     key_name            = var.key_name
7     associate_public_ip_address = true
8
9     tags = {
10         Name     = "${var.project_name}-bastion"
11         Project  = var.project_name
12         Role     = "bastion"
13     }
14 }
15
16 resource "aws_launch_template" "frontend_lt" {
17     name_prefix      = "${var.project_name}-fe-lt-"
18     image_id         = var.ami_id
19     instance_type    = "t2.micro"
20     key_name         = var.key_name
21     vpc_security_group_ids = [var.frontend_sg_id]
22
23     tag_specifications {
24         resource_type = "instance"
25         tags = {
26             Name     = "${var.project_name}-frontend"
27             Project  = var.project_name

```

```

28     Role      = "frontend"
29   }
30 }
31 }
32
33 resource "aws_autoscaling_group" "frontend_asg" {
34   name                  = "${var.project_name}-fe-asg"
35   vpc_zone_identifier = var.private_subnet_ids
36   min_size              = 2
37   max_size              = 4
38   desired_capacity     = 2
39
40   launch_template {
41     id      = aws_launch_template.frontend_lt.id
42     version = "$Latest"
43   }
44 }
45
46 resource "aws_launch_template" "backend_lt" {
47   name_prefix      = "${var.project_name}-be-lt-"
48   image_id         = var.ami_id
49   instance_type    = "t2.micro"
50   key_name         = var.key_name
51   vpc_security_group_ids = [var.backend_sg_id]
52
53   tag_specifications {
54     resource_type = "instance"
55     tags = {
56       Name      = "${var.project_name}-backend"
57       Project   = var.project_name
58       Role      = "backend"
59     }
60   }
61 }
62
63 resource "aws_autoscaling_group" "backend_asg" {
64   name                  = "${var.project_name}-be-asg"
65   vpc_zone_identifier = var.private_subnet_ids
66   min_size              = 2
67   max_size              = 4
68   desired_capacity     = 2
69
70   launch_template {
71     id      = aws_launch_template.backend_lt.id
72     version = "$Latest"
73   }
74 }
75
76 resource "aws_instance" "database" {
77   count      = 3
78   ami       = var.ami_id
79   instance_type = "t2.micro"
80   subnet_id   = var.private_subnet_ids[count.index % length(var.
     private_subnet_ids)]
81   vpc_security_group_ids = [var.db_sg_id]
82   key_name      = var.key_name
83 }

```

```

84 tags = {
85     Name      = "${var.project_name}-db-${count.index + 1}"
86     Project   = var.project_name
87     Role      = "database"
88 }
89 }

```

The root module wires the two ASGs to their respective ALB target groups using `aws_autoscaling_attachment` resources, ensuring that every new instance launched by an ASG scaling event is automatically registered with the appropriate Load Balancer and begins receiving traffic only once it passes the configured health check.

```

1 resource "aws_autoscaling_attachment" "asg_attachment_fe" {
2     autoscaling_group_name = module.compute.frontend_asg_name
3     lb_target_group_arn    = module.load_balancer.frontend_tg_arn
4 }
5
6 resource "aws_autoscaling_attachment" "asg_attachment_be" {
7     autoscaling_group_name = module.compute.backend_asg_name
8     lb_target_group_arn    = module.load_balancer.backend_tg_arn
9 }

```

7.1.2 variables.tf

All configurable values are extracted into input variables so that the module can be reused across environments. Security Group IDs are passed in from the Security Groups module outputs, and subnet IDs come from the Network module outputs, keeping each module's responsibilities cleanly separated.

```

1 variable "project_name" {}
2 variable "vpc_id" {}
3 variable "public_subnet_ids" { type = list(string) }
4 variable "private_subnet_ids" { type = list(string) }
5 variable "ami_id" { default = "ami-0c7217cdde317cfec" }
6 variable "instance_type" { default = "t2.micro" }
7 variable "key_name" {}
8 variable "bastion_sg_id" {
9     description = "Security Group ID for Bastion Host"
10    type        = string
11 }
12 variable "frontend_sg_id" {
13     description = "Security Group ID for Frontend servers"
14     type        = string
15 }
16 variable "backend_sg_id" {
17     description = "Security Group ID for Backend servers"
18     type        = string
19 }
20 variable "db_sg_id" {
21     description = "Security Group ID for Database servers"
22     type        = string
23 }

```

7.1.3 outputs.tf

The Bastion public IP is exported for use in Ansible inventory and SSH configuration. The three database private IPs are exported to allow Ansible to configure the MongoDB Replica Set. The two ASG names are exported to the root module so they can be referenced in the `aws_autoscaling_attachment` resources.

```

1 output "bastion_public_ip" {
2     description = "Public IP of the Bastion Host for SSH access"
3     value       = aws_instance.bastion.public_ip
4 }
5
6 output "database_private_ips" {
7     description = "Private IPs of the MongoDB instances for Replica
8     Set configuration"
9     value       = aws_instance.database[*].private_ip
10 }
11
12 output "frontend_asg_name" {
13     description = "Name of the Frontend Auto Scaling Group"
14     value       = aws_autoscaling_group.frontend_asg.name
15 }
16
17 output "backend_asg_name" {
18     description = "Name of the Backend Auto Scaling Group"
19     value       = aws_autoscaling_group.backend_asg.name
20 }

```

7.2 Deployed Compute Infrastructure

After running `terraform apply`, a total of **8 EC2 instances** were provisioned and reached a **Running** state inside VPC `vpc-06c3ce08596c4024a`. The table below summarises the complete deployed compute configuration for the `novolt` project.

Instance Name	Role	Type	Subnet	Public IP
novolt-bastion	Bastion	t2.micro	Public	3.231.160.248
novolt-frontend	Frontend	t2.micro	Private	—
novolt-frontend	Frontend	t2.micro	Private	—
novolt-backend	Backend	t2.micro	Private	—
novolt-backend	Backend	t2.micro	Private	—
novolt-db-1	Database	t2.micro	Private	—
novolt-db-2	Database	t2.micro	Private	—
novolt-db-3	Database	t2.micro	Private	—

ASG Name	Role	Desired	Min	Max
novolt-fe-asg	Frontend	2	2	4
novolt-be-asg	Backend	2	2	4

The Bastion Host is the only instance placed in a **public subnet** and assigned a public IP address (3.231.160.248). It serves as the single controlled SSH entry point into the VPC – all administrative access to the Frontend, Backend, and Database instances is routed through this host, which itself is protected by the `novolt-bastion-sg` Security Group.

The Frontend and Backend tiers are managed by their respective Auto Scaling Groups. Both ASGs span **two Availability Zones** (`us-east-1a` and `us-east-1b`), maintaining a desired

capacity of **2 instances** each and scaling up to a maximum of **4** under load. This ensures that the application remains available even if an entire Availability Zone becomes unreachable. New instances launched by a scaling event are automatically registered with the External and Internal ALB target groups respectively, and begin receiving traffic only after passing their health checks. The three Database instances are provisioned directly as `aws_instance` resources rather than through an ASG, because MongoDB Replica Set membership requires stable, known IP addresses for peer discovery and election. Instances are distributed across both private subnets using the `count.index % length(...)` pattern – two instances land in `us-east-1a` and one in `us-east-1b` – providing replication redundancy while keeping the primary election deterministic.

7.3 Verification Screenshots

The following screenshots were captured from the AWS Management Console to confirm that all compute resources were provisioned correctly by Terraform.

7.3.1 EC2 Instances

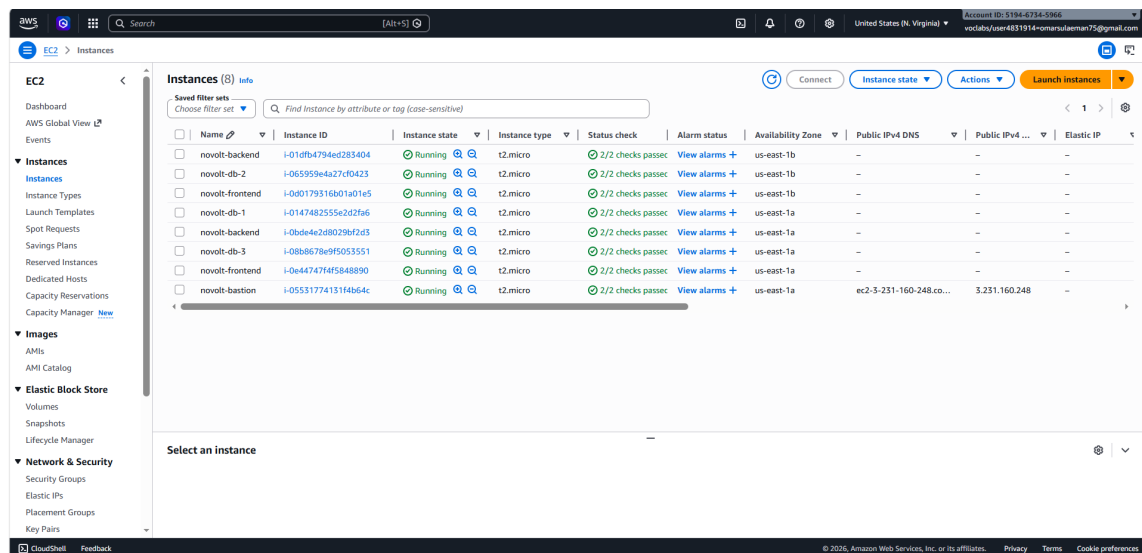


Figure 10: AWS Console – EC2 Instances: all 8 `novolt` instances in Running state, showing the Bastion Host with its public IP and all Frontend, Backend, and Database instances in private subnets

7.3.2 Auto Scaling Groups

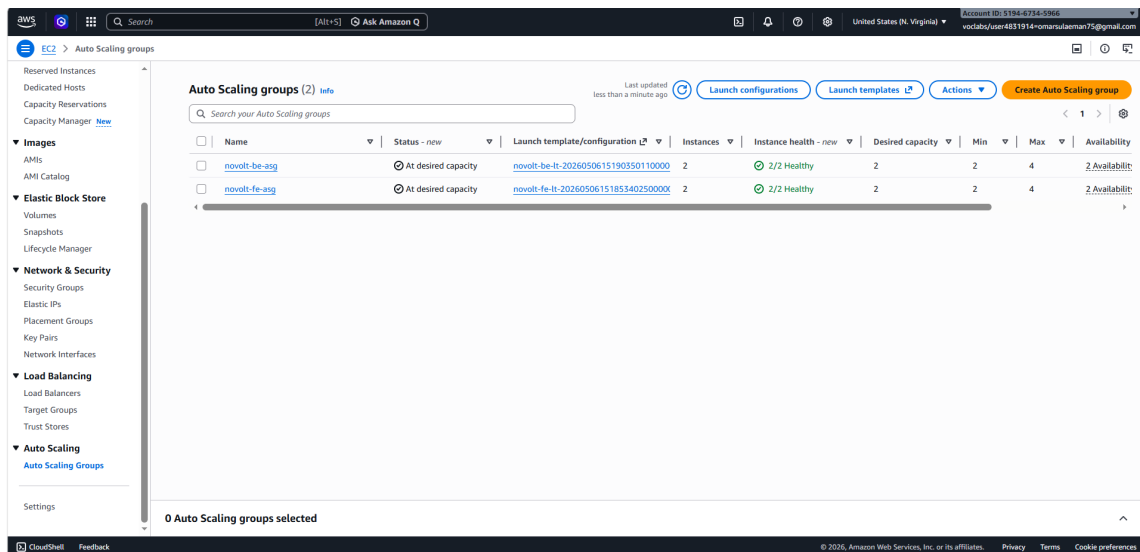


Figure 11: AWS Console – Auto Scaling Groups: `novolt-fe-asg` and `novolt-be-asg`, each with Desired 2 / Min 2 / Max 4, spanning two Availability Zones and attached to their respective ALB target groups

8 Ansible Configuration

Once Terraform has provisioned the full AWS infrastructure, **Ansible** takes over the configuration layer – installing software, deploying application code, and managing services across all three tiers. Rather than maintaining a static inventory file that would require manual updates every time the Auto Scaling Groups launch or terminate instances, this project uses **Dynamic Inventory** to automatically discover all running EC2 instances at playbook execution time. The configuration is organized as a set of **Ansible Roles** – one per tier – each encapsulating its own tasks, templates, and handlers. A single top-level playbook, `site.yml`, orchestrates the execution order: Database servers are configured first so that the Replica Set is ready before the Backend attempts to connect, followed by Backend servers, and finally Frontend servers. All roles run with elevated privileges via `become: yes`.

8.1 Dynamic Inventory

Static inventory files are brittle in autoscaled environments – every scaling event changes the set of active instances. This project solves that problem using the **AWS EC2 Dynamic Inventory plugin** (`amazon.aws.aws_ec2`), which queries the AWS API at runtime and builds a live inventory of all instances matching the `Project: novolt` tag.

Instances are automatically grouped by their `Role` tag into `role_database`, `role_backend`, and `role_frontend` groups, exactly matching the group names referenced in `site.yml` and the role tasks. Because all Backend and Database instances are in private subnets with no public IP addresses, Ansible reaches them by tunnelling every SSH connection through the Bastion Host using a `ProxyCommand` configured in `ansible.cfg`.

8.1.1 aws_ec2.yml

```
1 plugin: amazon.aws.aws_ec2
```

```

2 regions:
3   - us-east-1
4 filters:
5   "tag:Project": novolt
6 keyed_groups:
7   - key: tags.Role
8     prefix: role
9     separator: "_"
10 hostnames:
11   - private-ip-address
12 compose:
13   ansible_host: private_ip_address

```

The `keyed_groups` block instructs the plugin to create an inventory group for each unique value of the Role EC2 tag, prefixed with `role_`. Setting `hostnames` to `private-ip-address` and composing `ansible_host` from the private IP ensures that Ansible connects to the correct internal address, relying on the Bastion proxy for transport rather than attempting a direct connection.

8.1.2 ansible.cfg

```

1 [defaults]
2 inventory          = aws_ec2.yml
3 host_key_checking  = False
4 remote_user        = ubuntu
5 private_key_file   = ~/vockey.pem
6
7 [ssh_connection]
8 ssh_args = -o ProxyCommand="ssh -W %h:%p -q ubuntu@3.231.160.248 -i
   ~/vockey.pem"

```

The `ProxyCommand` in `[ssh_connection]` instructs Ansible to forward every SSH connection through the Bastion Host at 3.231.160.248. The `-W %h:%p` flag tells the Bastion to transparently relay the TCP stream to the target private IP and port, so Ansible communicates end-to-end as if it had a direct connection to each private instance – without ever exposing those instances to the public internet.

8.1.3 group_vars/role_frontend.yml

```

1 backend_alb_dns: "internal-novolt-int-alb-2100570306.us-east-1.elb.
   amazonaws.com"

```

The Internal ALB DNS name is injected into the Frontend role as a group variable. This value is rendered into the Nginx configuration template at playbook execution time, allowing the Frontend servers to proxy API requests to the Backend tier through the Internal ALB without any hard-coded IP addresses.

8.2 Main Playbook

The top-level playbook `site.yml` defines the execution order for all three roles. The Database play runs first and is intentionally placed before the Backend play because the Backend's `.env` file is generated with the MongoDB connection string during configuration – if the Replica Set were not yet initialized, the Backend application would fail to start.

```

1 ---
2 - name: Configure Database Servers

```

```

3  hosts: role_database
4  become: yes
5  roles:
6    - database
7
8  - name: Configure Backend Servers
9    hosts: role_backend
10   become: yes
11   roles:
12     - backend
13
14  - name: Configure Frontend Servers
15    hosts: role_frontend
16    become: yes
17    roles:
18      - frontend

```

8.3 Database Role

The Database role installs **MongoDB 7.0** on all three `role_database` instances, deploys a custom `mongod.conf` via a Jinja2 template, and initializes a **three-member Replica Set** named `rs0`. The Replica Set initialization task uses `run_once: true` so that `rs.initiate()` is called exactly once – from the first database host – with all three members declared in a single atomic operation.

8.3.1 templates/mongod.conf.j2

```

1  storage:
2    dbPath: /var/lib/mongodb
3
4  systemLog:
5    destination: file
6    logAppend: true
7    path: /var/log/mongodb/mongod.log
8
9  net:
10   port: 27017
11   bindIp: 0.0.0.0
12
13  processManagement:
14    timeZoneInfo: /usr/share/zoneinfo
15
16  replication:
17    replSetName: "rs0"

```

Setting `bindIp: 0.0.0.0` allows MongoDB to accept connections on all network interfaces within the instance. Access is restricted exclusively at the Security Group level – only the Backend Security Group and the `self` rule for intra-replica-set traffic are permitted on port 27017, as defined in the `novolt-db-sg` Security Group.

8.3.2 tasks/main.yml

```

1  ---
2  - name: Import MongoDB public GPG key
3    apt_key:

```

```

4     url: https://pgp.mongodb.com/server-7.0.asc
5     state: present
6
7 - name: Add MongoDB repository
8   apt_repository:
9     repo: "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/
10          ubuntu
11          jammy/mongodb-org/7.0 multiverse"
12     state: present
13
14 - name: Install MongoDB
15   apt:
16     name: mongodb-org
17     state: present
18     update_cache: yes
19
20 - name: Configure MongoDB (mongod.conf)
21   template:
22     src: mongod.conf.j2
23     dest: /etc/mongod.conf
24     notify: Restart MongoDB
25
26 - name: Start and enable MongoDB service
27   service:
28     name: mongod
29     state: started
30     enabled: yes
31
32 - name: Wait for MongoDB to be ready (30 seconds)
33   pause:
34     seconds: 30
35
36 - name: Initialize MongoDB Replica Set
37   shell: |
38     mongosh --eval 'rs.initiate({
39       _id: "rs0",
40       members: [
41         { _id: 0, host: "{{ hostvars[groups["role_database"][0]]
42                  ["private_ip_address"] }}:27017"},
43         { _id: 1, host: "{{ hostvars[groups["role_database"][1]]
44                  ["private_ip_address"] }}:27017"},
45         { _id: 2, host: "{{ hostvars[groups["role_database"][2]]
46                  ["private_ip_address"] }}:27017"}
47       ]
48     })'
49   run_once: true
50   register: rs_init_result

```

8.3.3 handlers/main.yml

```

1 ---
2 - name: Restart MongoDB
3   service:
4     name: mongod
5     state: restarted

```

8.4 Backend Role

The Backend role installs **Node.js 18.x** and **PM2** on all `role_backend` instances, clones the application repository, and starts the API server as a managed PM2 process. The MongoDB connection string is generated dynamically by interpolating the three database instance private IPs – sourced from the dynamic inventory's `hostvars` – directly into the `.env` file, producing a full Replica Set URI with no hard-coded addresses.

8.4.1 tasks/main.yml

```

1  ---
2  - name: Install standard dependencies
3    apt:
4      name: ['git', 'curl']
5      state: present
6      update_cache: yes
7
8  - name: Add NodeSource repository for Node.js 18.x
9    shell: curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E
10         bash -
11
12 - name: Install Node.js
13   apt:
14     name: nodejs
15     state: present
16
17 - name: Install PM2 globally
18   command: npm install -g pm2
19
20 - name: Clone the novolt-waitlist repository
21   git:
22     repo: 'https://github.com/OmarAlaaEl-Din/novolt-waitlist.git'
23     dest: /home/ubuntu/novolt-waitlist
24     force: yes
25
26 - name: Install NPM packages
27   command: npm install
28   args:
29     chdir: /home/ubuntu/novolt-waitlist/api
30
31 - name: Create .env file with Dynamic Database URI
32   copy:
33     dest: /home/ubuntu/novolt-waitlist/api/.env
34     content: |
35       PORT=3000
36       MONGO_URI=mongodb://
37         {{- hostvars[groups['role_database'][0]]['private_ip_address'] }}:27017,
38         {{- hostvars[groups['role_database'][1]]['private_ip_address'] }}:27017,
39         {{- hostvars[groups['role_database'][2]]['private_ip_address'] }}:27017
40       /novolt?replicaSet=rs0
41
42 - name: Restart backend app with PM2
43   shell: "pm2 restart novolt-backend || pm2 start index.js --name
44         novolt-backend"

```

```

43     args:
44         chdir: /home/ubuntu/novolt-waitlist/api
45
46 - name: Save PM2 config to start on boot
47     command: pm2 save

```

The `pm2 restart novolt-backend || pm2 start ...` idiom makes the task idempotent: if the process is already running from a previous playbook run, PM2 restarts it cleanly; if it does not yet exist, PM2 starts it for the first time. The subsequent `pm2 save` persists the process list so that the Backend application survives instance reboots without manual intervention.

8.5 Frontend Role

The Frontend role installs **Nginx** on all `role_frontend` instances, deploys the static web assets cloned from the application repository, and configures Nginx as a **reverse proxy**. All requests to the `/api/` path are forwarded to the Internal ALB DNS name injected via the `backend_alb_dns` group variable, while all other requests are served directly from the local filesystem.

8.5.1 templates/nginx_conf.j2

```

1  server {
2      listen 80;
3      root /var/www/html;
4      index index.html;
5
6      location / {
7          try_files $uri $uri/ =404;
8      }
9
10     location /api/ {
11         proxy_pass http://{{ backend_alb_dns }};
12         proxy_set_header Host $host;
13         proxy_set_header X-Real-IP $remote_addr;
14     }
15 }

```

8.5.2 tasks/main.yml

```

1  ---
2  - name: Install Nginx
3      apt:
4          name: nginx
5          state: present
6          update_cache: yes
7
8  - name: Clone the repository for Frontend files
9      git:
10         repo: 'https://github.com/OmarAlaaEl-Din/novolt-waitlist.git'
11         dest: /tmp/novolt-repo
12         force: yes
13
14 - name: Copy frontend files to Nginx web root
15     copy:
16         src: /tmp/novolt-repo/
17         dest: /var/www/html/

```

```
18     remote_src: yes
19
20 - name: Configure Nginx as Reverse Proxy
21   template:
22     src: nginx_conf.j2
23     dest: /etc/nginx/sites-available/default
24     notify: Restart Nginx
25
26 - name: Start and Enable Nginx
27   service:
28     name: nginx
29     state: started
30     enabled: yes
```

8.5.3 handlers/main.yml

```
1 ---
2 - name: Restart Nginx
3   service:
4     name: nginx
5     state: restarted
```

The handler-based restart pattern ensures that Nginx is only reloaded when the configuration template actually changes. If the **Configure Nginx as Reverse Proxy** task reports no change – because the rendered template is identical to the file already on disk – the handler is never triggered, making the playbook safe to run repeatedly without causing unnecessary service disruptions.

8.6 Verification Screenshots

The following screenshot was captured from the terminal after executing **ansible-playbook site.yml**, confirming that the full configuration pipeline completed successfully across all three tiers with zero failures.

8.6.1 Playbook Execution Summary

```

root@omar:~/novolt-project/ansible
ok: [10.0.4.190]
ok: [10.0.3.198]

TASK [Backend : Restart backend app with PM2] *****
changed: [10.0.3.198]
changed: [10.0.4.190]

TASK [Backend : Save PM2 config to start on boot] *****
changed: [10.0.4.190]
changed: [10.0.3.198]

PLAY [Configure Frontend Servers] *****

TASK [Gathering Facts] *****
ok: [10.0.4.210]
ok: [10.0.3.201]

TASK [Frontend : Install Nginx] *****
ok: [10.0.3.201]
ok: [10.0.4.210]

TASK [Frontend : Clone the repository for Frontend files] *****
ok: [10.0.4.210]
ok: [10.0.3.201]

TASK [Frontend : Copy frontend files to Nginx directory] *****
changed: [10.0.3.201]
changed: [10.0.4.210]

TASK [Frontend : Configure Nginx as Reverse Proxy] *****
ok: [10.0.3.201]
ok: [10.0.4.210]

TASK [Frontend : Start and Enable Nginx] *****
ok: [10.0.4.210]
ok: [10.0.3.201]

PLAY RECAP *****
10.0.3.123      : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.3.198      : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.3.201      : ok=6  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.3.22       : ok=6  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.4.190      : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.4.210      : ok=6  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
10.0.4.62       : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

[root@omar ansible]#

```

Figure 12: Ansible **PLAY RECAP** – `site.yml` executed against all 7 managed hosts. The Frontend play shows Nginx installed, the repository cloned, frontend files copied, and Nginx configured as a reverse proxy across both Frontend instances (10.0.3.201, 10.0.4.210). The Backend play confirms PM2 restarted and its config saved across both Backend instances (10.0.3.198, 10.0.4.190). The final **PLAY RECAP** shows all 7 hosts – 10.0.3.123, 10.0.3.198, 10.0.3.201, 10.0.3.22, 10.0.4.190, 10.0.4.210, 10.0.4.62 – with `unreachable=0` and `failed=0` across the board, confirming a fully successful run.

9 End-to-End Testing

With the full infrastructure provisioned by Terraform and all three tiers configured by Ansible, the final step is to verify that the application works correctly from end to end – from a user submitting their email in the browser all the way through to the data being persisted and replicated across all three MongoDB nodes.

The application is accessed via the **External ALB** DNS name which serves as the single public entry point. The ALB forwards the request to one of the two healthy Frontend instances, which in turn proxies API calls through the Internal ALB to the Backend, which writes to the MongoDB Replica Set. All three tests below confirm that this full chain is functioning correctly.

9.1 Test 1: Application Loads Successfully

The first test confirms that the **Novolt** web application is reachable from the internet via the External ALB and that the Frontend is serving the static assets correctly. Navigating to the ALB DNS name in a browser loads the full landing page – including the hero text, the Available Bays widget, the waitlist signup form, and the QR code – with no errors.

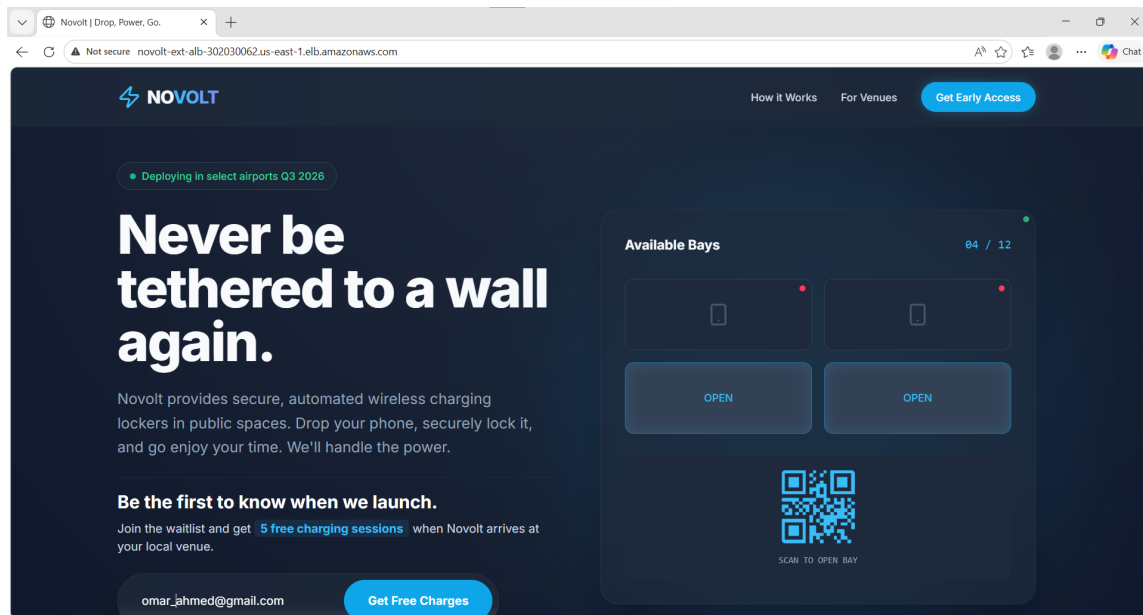


Figure 13: The Novolt web application loading successfully in a browser via the External ALB DNS name `novolt-ext-alb-302030062.us-east-1.elb.amazonaws.com`, confirming that the Frontend Nginx servers are running, the External ALB health checks are passing, and static assets are being served correctly

9.2 Test 2: Waitlist Signup Works End-to-End

The second test verifies the full request path through all three tiers. A test email address (`omar_ahmed@gmail.com`) was submitted through the waitlist signup form on the landing page. The Backend API processed the request, wrote the new entry to the MongoDB Replica Set Primary, and returned a success response. The Frontend then displayed the confirmation message “*You’re on the list! We’ll notify you when we launch.*” – confirming that the Frontend, Internal ALB, Backend, and Database are all communicating correctly.

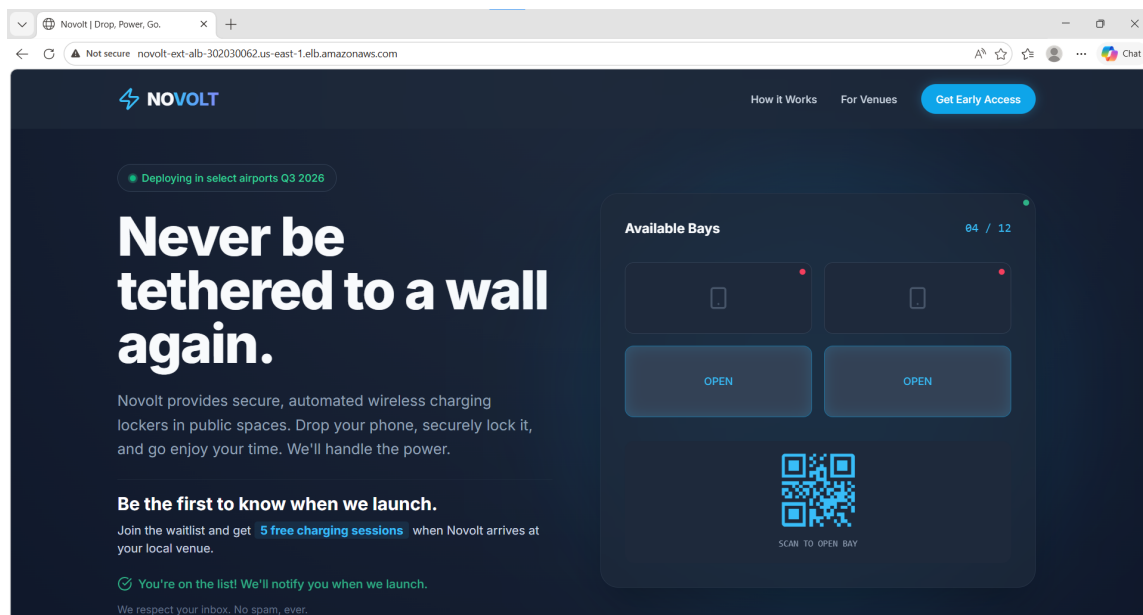


Figure 14: Successful waitlist signup – after submitting `omar_ahmed@gmail.com` through the form, the Frontend displays the confirmation message “You’re on the list! We’ll notify you when we launch.”, confirming end-to-end connectivity across all three application tiers via the External ALB, Internal ALB, Backend API, and MongoDB Replica Set

9.3 Test 3: Data Replicated Across All MongoDB Nodes

The third test confirms that MongoDB Replica Set replication is working correctly. Using Ansible’s ad-hoc command mode, a `mongosh` query was run against all three database instances simultaneously to retrieve the contents of the `waitlists` collection:

```
1 ansible role_database -b -m shell \
2   -a "mongosh novolt --eval 'db.waitlists.find()'"
```

All three nodes – 10.0.3.22, 10.0.3.123, and 10.0.4.62 – returned the same document: the entry for `omar_ahmed@gmail.com` with `ObjectId('69fb886fe2f22729fe24730f')` and `signup_date: ISODate('2026-05-06T18:29:03.025Z')`. This confirms that the write performed in Test 2 was successfully replicated from the Primary to both Secondary nodes, and that the Replica Set is operating as expected.

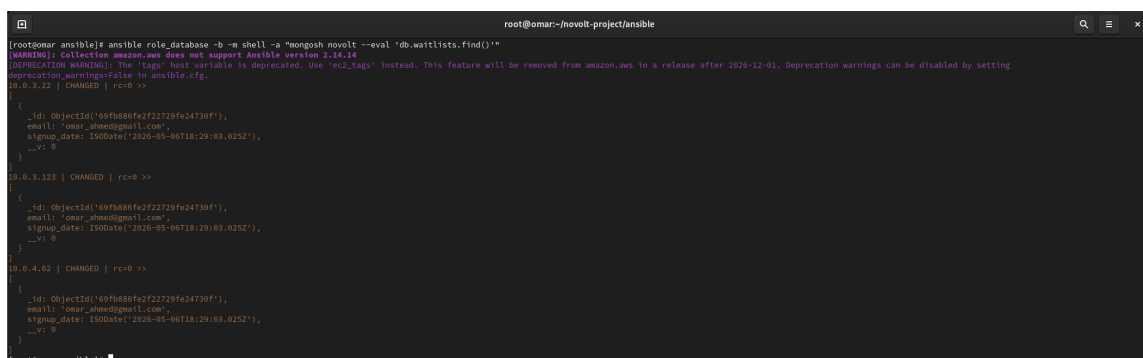


Figure 15: Ansible ad-hoc `mongosh` query run against all three `role_database` hosts (10.0.3.22, 10.0.3.123, 10.0.4.62), each returning the identical waitlist document for `omar_ahmed@gmail.com` – confirming that MongoDB Replica Set replication across all three nodes is functioning correctly

9.4 Summary

All three tests passed without any issues. The results confirm that the complete three-tier architecture deployed by Terraform and configured by Ansible is fully operational:

- **Frontend** – Nginx is running on both instances, serving the application through the External ALB with health checks passing.
- **Backend** – The Node.js API is running under PM2 on both Backend instances, reachable from the Frontend via the Internal ALB, and successfully processing write operations.
- **Database** – The MongoDB Replica Set `rs0` is operational across all three nodes, with writes from the Primary automatically replicated to both Secondary members.

The infrastructure can be fully reproduced at any time by running `terraform apply` followed by `ansible-playbook site.yml`, with no manual steps required.